Национальный исследовательский университет "Высшая школа экономики"
Факультет компьютерных наук
Образовательная программа "Программная инженерия"

СОГЛАСОВАНО

Старший преподаватель, Департамент анализа данных и искусственного интеллекта, НИУ ВШЭ

_____ Паринов Андрей Александрович

«___» _____ 2026 г.

УТВЕРЖДЕНО

Академический руководитель образовательной программы "Программная инженерия", старший преподаватель департамента программной инженерии

_____ Н. А. Павлочев

«___» _____ 2026 г.

ЦИФРОВОЙ АССИСТЕНТ СТУДЕНТА: КЛИЕНТСКАЯ ЧАСТЬ

Пояснительная записка

ЛИСТ УТВЕРЖДЕНИЯ

RU.17701729.12.17 81 01-1-ЛУ

Исполнители:

Студент группы БПИ236

_____ / Радионов Никита Дмитриевич /

«___» _____ 2026 г.

2026

Подп. и дата	
Инв.№ дубл.	
Взам. инв.№	
Подп. и дата	
Инв.№ подп	

УТВЕРЖДЕН

RU.17701729.12.17 81 01-1-ЛУ

ЦИФРОВОЙ АССИСТЕНТ СТУДЕНТА: КЛИЕНТСКАЯ ЧАСТЬ

Пояснительная записка

RU.17701729.12.17 81 01-1

Листов 18

2026

Инв.№ подп	Подп. и дата	Взам. инв.№	Инв.№ дубл.	Подп. и дата

АННОТАЦИЯ

Данный программный документ представляет собой пояснительную записку к программному проекту «Цифровой Ассистент Студента: клиентская часть» содержит следующие разделы: «Введение», «Назначение и область применения», «Технические характеристики», «Ожидаемые технико-экономические показатели», приложения.

Раздел «Введение» включает в себя наименование программы и документ, на основании которого ведётся разработка, с указанием организации, утвердившей данный документ.

В разделе «Назначение и область применения» содержатся функциональное и эксплуатационное назначение программы и краткая характеристика области её применения.

В разделе «Назначение разработки» указано функциональное и эксплуатационное назначение создаваемого программного продукта.

В разделе «Технические характеристики» присутствуют следующие подразделы: постановка задачи на разработку программы, описание функционирования программы, описание и обоснование алгоритма работы программы, описание и обоснование выбора метода организации входных и выходных данных, описание работы с базой данных, описание и обоснование выбора состава технических и программных средств.

Раздел «Требования к программным документам» содержит указание на предварительный состав программной документации и специальные требования к ней.

В разделе «Ожидаемые технико-экономические показатели» указана предполагаемая потребность и экономические преимущества разработки по сравнению с отечественными и зарубежными образцами или аналогами.

Настоящий документ разработан в соответствии с требованиями:

1. ГОСТ 19.101-77 [1]: Виды программ и программных документов.
2. ГОСТ 19.102-77 [2]: Стадии разработки.
3. ГОСТ 19.103-77 [3]: Обозначения программ и программных документов.
4. ГОСТ 19.104-78 [4]: Основные надписи.
5. ГОСТ 19.105-78 [5]: Общие требования к программным документам.
6. ГОСТ 19.106-78 [6]: Требования к программным документам, выполненным печатным способом.
7. ГОСТ 19.404-79 [10]: Пояснительная записка. Требования к содержанию и оформлению.

Изменения к данной Пояснительной записке оформляются согласно ГОСТ 19.603-78 [12], ГОСТ 19.604-78 [13].

СОДЕРЖАНИЕ

1. ВВЕДЕНИЕ 4
1.1. Наименование программы 4
1.2. Документ(ы), на основании которого(ых) ведется разработка 4
2. НАЗНАЧЕНИЕ РАЗРАБОТКИ 5
2.1. Функциональное назначение 5
2.2. Эксплуатационное назначение 5
2.3. Краткая характеристика области применения программы 5
3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ 6
3.1. Постановка задачи на разработку программы 6
3.2. Описание и обоснование выбора состава технических и программных средст 7
3.2.1. Состав технических и программных средств 7
3.2.2. Обоснование выбора технических и программных средств 7
3.3. Описание и обоснование архитектуры программы 8
3.4. Алгоритм работы программы 9
3.4.1. Аутентификация и авторизация 9
3.4.2. Обработка запроса к каталогу 9
3.4.3. Получение рекомендаций 9
3.4.4. Управление заявками 10
3.5. Особенности функционала 10
3.5.1. Каталог проектов (/projects/) 10
3.5.2. Кабинет заказчика (/projects/) 10
3.5.3. Подача заявки (/projects/ 11
3.5.4. Форма проекта (/projects/create/) 11
3.5.5. Кабинет ЦППРП (/cpprp/) 11
3.5.6. Профиль (/profile/) 11
3.6. Обоснование выбора метода организации входных и выходных данных 12
4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ 13
4.1. Ориентировочная экономическая эффективность 13
4.2. Предполагаемая потребность 13
4.3. Экономические преимущества разработки по сравнению с отечественными и зарубежными аналогами 13
ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ 15
ПРИЛОЖЕНИЕ 2. ССЫЛКИ НА АНАЛОГИ 16
ПРИЛОЖЕНИЕ 3. ТЕРМИНОЛОГИЯ 17

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование программы – «Цифровой Ассистент Студента: клиентская часть».

Наименование программы на английском языке – «Digital Student Assistant: Client Side».

1.2. Документ(ы), на основании которого(ых) ведется разработка

Разработка ведется на основании учебного плана подготовки бакалавров по направлению 09.03.04 «Программная инженерия» и утвержденной академическим руководителем программы темы курсового проекта. Документ утверждён организацией: Национальный исследовательский университет «Высшая школа экономики», Факультет компьютерных наук, образовательная программа «Программная инженерия».

2. НАЗНАЧЕНИЕ РАЗРАБОТКИ

2.1. Функциональное назначение

Клиентская часть Цифрового Ассистента Студента обеспечивает интерфейс взаимодействия пользователей с системой управления проектной деятельностью. Программа реализует SSR-рендеринг страниц с использованием Django 5 и Tailwind CSS, предоставляя функционал для трёх ролей: студентов (просмотр каталога, подача заявок), заказчиков (управление проектами) и модераторов ЦППРП (администрирование платформы). Ключевые функции включают персонализированные рекомендации проектов через ML-сервис с автоматическим fallback на keyword-поиск, оптимистичное обновление интерфейса при работе с закладками и полный цикл модерации проектов.

2.2. Эксплуатационное назначение

Программа предназначена для сотрудников и студентов НИУ ВШЭ, участвующих в проектной деятельности. Целевая аудитория: студенты 2–4 курсов ФКН (до 1700 пользователей в пиковые периоды набора), преподаватели и внешние заказчики проектов (до 200 активных пользователей) и сотрудники ЦППРП, выполняющие функции модерации (5–10 человек). Требуется базовая компьютерная грамотность и доступ к современному веб-браузеру. Специальная техническая подготовка не предусмотрена — интерфейс оптимизирован для пользователей без профильного IT-образования.

2.3. Краткая характеристика области применения программы

Разработка применяется в образовательном процессе НИУ ВШЭ для организации проектной деятельности студентов. Программа автоматизирует ключевые процессы: поиск и подбор проектов по компетенциям, обработку заявок, формирование команд и контроль дедлайнов. Основные бизнес-процессы включают публикацию проектов преподавателями, модерацию со стороны ЦППРП, подачу заявок студентами и формирование отчетов. Технически система интегрирована с SSO edu.hse.ru и обеспечивает экспорт данных в форматах, совместимых с LMS университета.

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку программы

Целью разработки является создание серверно-рендеримой (SSR) клиентской части веб-платформы «Цифровой Ассистент Студента: клиентская часть», обеспечивающей полный набор пользовательских сценариев для трёх ролей: студент, заказчик (преподаватель) и модератор (ЦППРП). Ключевые ограничения разработки: интеграция с ML-сервисом рекомендаций (FastAPI, порт 8001) при обеспечении отказоустойчивости через fallback-алгоритм, время отклика интерфейса не более 3 секунд. Ожидаемый результат — SSR веб-приложение на Django 5 и HTMX 1.9 с полным циклом пользовательских сценариев согласно требованиям технического задания.

В рамках разработки поставлены и решены следующие задачи.

Проектирование экранов и ролевой навигации. Разработана система экранов, разграничивающая функциональность по ролям пользователя. Для студента реализован каталог проектов с четырьмя вкладками («Все проекты», «Рекомендации», «Закладки», «Мои заявки») и персональный кабинет с обзором статистики, ближайших дедлайнов и доступных шаблонов документов. Для заказчика реализован кабинет с пятью вкладками («Сводка», «Мои проекты», «Статьи», «Сотрудники», «Граф соавторства»). Для модератора — расширенный административный кабинет с пятью функциональными вкладками. Доступ к каждой группе экранов защищён декораторами login_required, moderator_required и student_required.

Реализация динамических обновлений интерфейса. Для обеспечения отзывчивости интерфейса без перехода к SPA-архитектуре применена библиотека HTMX версии 1.9. Поиск и фильтрация в каталоге реализованы с debounce-задержкой 350 мс: обновляется только сетка карточек, а URL изменяется через атрибут hx-push-url, что обеспечивает корректную работу кнопки «Назад» и прямых ссылок на отфильтрованные выборки. Закладки обновляются оптимистично — иконка меняется мгновенно, откат выполняется только при получении ошибки от сервера.

Интеграция с ML-сервисом рекомендаций. Реализовано взаимодействие с ML-сервисом на FastAPI для получения персонализированных рекомендаций проектов на основе интересов пользователя из профиля. При недоступности сервиса (исключения ConnectionError или таймаут) активируется резервный алгоритм подбора по совпадению тегов с интересами через Q-запросы Django ORM, что гарантирует непрерывность функции рекомендаций.

Реализация визуализации графа соавторства. В кабинете преподавателя реализована вкладка «Граф соавторства» на основе библиотеки Vis.js версии 9.1.9. Граф отображает авторов публикаций в виде узлов — размер узла пропорционален числу публикаций автора — и рёбра между соавторами совместных работ. Реализована ленивая инициализация: Canvas создаётся только при первом открытии вкладки, что исключает неиспользуемые вычисления на других вкладках.

Разработка дашборда заказчика и кабинета ЦППРП. Для роли заказчика реализована сводная вкладка кабинета, агрегирующая ключевые метрики через ORM-аннотации без избыточных запросов: число опубликованных проектов, количество заявок по статусам, свободные места в командах. Для сотрудников ЦППРП разработан отдельный административный кабинет: управление дедлайнами и шаблонами документов, просмотр всех заявок с фильтрацией по статусу, экспорт данных в CSV с кодировкой UTF-8 BOM для совместимости с Microsoft Excel.

Реализация справочника технологий. Разработан публичный справочник одобренных технологических тегов с отображением числа активных проектов по каждому тегу. Каждый тег служит навигационной точкой входа: нажатие открывает каталог проектов с соответствующим активным фильтром. Сотрудники ЦППРП одобряют или отклоняют новые теги из очереди ожидающих рассмотрения непосредственно из справочника.

Реализация валидации форм и автоматизированных тестов. Реализована многоуровневая валидация пользовательских форм. Серверная валидация обеспечивает проверку минимальной длины мотивационного письма (не менее 30 символов), корректности тегов интересов и технологий по регулярному выражению, минимальной и максимальной длины отдельного тега. Клиентская валидация реализована через атрибут minlength и JavaScript-счётчики символов с визуальной обратной связью. Написан комплект из 17 автоматизированных тестов на pytest-django, охватывающих доступность страниц, ролевое разграничение, работу закладок и валидацию форм.

3.2. Описание и обоснование выбора состава технических и программных средст

3.2.1. Состав технических и программных средств

Серверное оборудование: процессор с четырьмя и более ядрами с тактовой частотой от 2,5 ГГц, оперативная память не менее 8 ГБ, SSD-накопитель от 100 ГБ. Клиентское оборудование: процессор с двумя и более ядрами, оперативная память от 4 ГБ, современный браузер (Chrome 90+, Firefox 90+, Edge 90+). Специальная установка программного обеспечения на стороне клиента не требуется.

Серверный программный стек: Python 3.12, Django 5, HTMX 1.9.12, Tailwind CSS v2.2.19, PostgreSQL 16, Vis.js 9.1.9. Инструменты разработки и тестирования: pytest-django (17 тестов), Docker Compose, uv (менеджер зависимостей и виртуальных окружений). Внешние интегрируемые сервисы: FastAPI ML-сервис (порт 8001), SSO edu.hse.ru, eLIBRARY/Scopus API.

3.2.2. Обоснование выбора технических и программных средств

Django выбран как полнофункциональный веб-фреймворк с встроенными механизмами аутентификации, ORM, системой миграций, CSRF-защитой и безопасным экранированием в шаблонах. Модульная архитектура Django позволяет организовать клиентскую часть как независимый модуль apps/frontend, работающий с общей базой данных напрямую — без избыточных HTTP-запросов между компонентами одного процесса, что обеспечивает транзакционность операций и упрощает отладку.

HTMX выбран для реализации динамических обновлений интерфейса без перехода к SPA-архитектуре с отдельным build-процессом. Библиотека позволяет описывать динамическое поведение декларативно через HTML-атрибуты (hx-get, hx-post, hx-swap, hx-target), что значительно снижает объём клиентского JavaScript-кода. Серверные обработчики возвращают HTML-фрагменты (partials) вместо полных страниц, обеспечивая отзывчивость без дополнительного слоя управления состоянием.

Tailwind CSS v2 выбран как утилитарный CSS-фреймворк, позволяющий формировать стиль компонентов непосредственно в HTML-разметке без создания отдельных CSS-файлов и без

настройки сборочного шага. Это ускоряет разработку и устраняет конфликты между стилями разных частей приложения.

PostgreSQL выбран как основная СУБД в соответствии с требованиями общего технического задания. Django ORM обеспечивает работу с базой данных через Python-объекты и поддерживает аннотации Count(), F()-выражения и сложные JOIN-запросы, необходимые при агрегации статистики в дашбордах заказчика и модератора.

Vis.js 9.1.9 выбран для визуализации графа соавторства как зрелая библиотека с поддержкой физического моделирования (алгоритм Barnes-Hut), интерактивных тултипов и программного управления узлами и рёбрами. Библиотека подключается через CDN без сборочного шага, что упрощает интеграцию в Django-шаблон.

Python 3.12 обеспечивает долгосрочную поддержку до 2028 года, современные языковые возможности и единый язык для всех компонентов клиентской части. FastAPI на стороне ML-сервиса обеспечивает высокую производительность асинхронной обработки запросов рекомендаций благодаря ASGI-архитектуре.

3.3. Описание и обоснование архитектуры программы

Клиентская часть реализована по паттерну Model-View-Template (MVT) — производной от классического MVC, адаптированной под особенности Django. Сервер обрабатывает HTTP-запросы, формирует контекст через Django ORM и передаёт его в шаблоны, которые компилируются в HTML на стороне сервера и отправляются браузеру в готовом виде (Server-Side Rendering, SSR). Такой подход обеспечивает быстрое время первоначальной загрузки, корректную работу на устройствах с отключённым JavaScript и простую интеграцию с системой ролей, форм и CSRF-защитой Django.

Архитектурно фронтенд-приложение оформлено как самостоятельный Django-модуль (apps/frontend), использующий общую базу данных с серверной частью напрямую через ORM, минуя REST API. Это позволяет избежать избыточных HTTP-запросов внутри одного процесса и обеспечивает транзакционность операций. Модуль views разделён на доменные файлы: projects.py, applications.py, profile.py, technologies.py, student.py, cpprp.py. Кастомные теги шаблонов вынесены в директорию templatetags/, что обеспечивает возможность повторного использования фрагментов разметки.

Для частичных обновлений страниц применяется HTMX. При наличии заголовка HX-Request в запросе view-функция возвращает только HTML-фрагмент (partial) — часть страницы для обновления — вместо полного ответа. Атрибут hx-push-url обновляет URL в адресной строке без перезагрузки, обеспечивая корректную работу кнопки «Назад» и сохранение фильтров в прямых ссылках. Операции с закладками возвращают JSON-ответ, обрабатываемый JavaScript для оптимистичного обновления UI.

Структурно система включает четыре слоя: слой представления (Django views) — обрабатывает HTTP-запросы и формирует контекст; слой шаблонов — генерирует HTML из контекста с использованием Django Template Language; слой прикладной логики — переходы статусов заявок через модуль transitions, проверки прав доступа, сценарии модерации; слой инфраструктуры — работа с PostgreSQL, внешними API и генерация файлов экспорта.

Интеграция с ML-сервисом реализована как синхронный HTTP-запрос из view-функции с явной обработкой исключений: при ConnectionError или превышении таймаута активируется fallback — поиск через Q-запросы ORM по совпадению тегов с интересами пользователя. Такой механизм обеспечивает устойчивость к частичным сбоям: клиентская часть предоставляет рекомендации даже при полной недоступности ML-сервиса.

3.4. Алгоритм работы программы

3.4.1. Аутентификация и авторизация

При обращении к защищённым страницам применяется декоратор login_required, перенаправляющий неавторизованных пользователей на страницу входа /auth/. После успешной аутентификации через SSO edu.hse.ru Django создаёт сессию; идентификатор хранится в защищённой httponly-куке. При каждом последующем запросе сессия восстанавливается автоматически без повторного обращения к SSO.

Роль пользователя определяется по полю role объекта UserProfile, связанного с моделью User через отношение OneToOne. На основе роли view-функции формируют различный контекст и выбирают соответствующий шаблон или ветку логики. Для ограничения доступа к административным функциям ЦППРП применяется декоратор moderator_required; для функций, доступных только студентам, — student_required. При нарушении ролевых ограничений возвращается ответ с кодом 403.

3.4.2. Обработка запроса к каталогу

При поступлении GET-запроса на /projects/ view-функция определяет роль пользователя. Для студента и модератора выполняется выборка проектов из базы данных с учётом параметров фильтрации: теги технологий (tech_tags), размер команды (team_size), текстовый поиск (q). Каждый параметр применяется как условие filter() к QuerySet; несколько параметров комбинируются через AND.

При наличии заголовка HX-Request view возвращает только HTML-фрагмент сетки карточек (partial), иначе — полную страницу с базовым шаблоном. Для заказчика view возвращает сводный дашборд с агрегированными счётчиками статусов и список собственных проектов, аннотированных количеством ожидающих рассмотрения заявок (pending_count).

3.4.3. Получение рекомендаций

При открытии вкладки «Рекомендации» view-функция извлекает интересы пользователя из профиля (UserProfile.interests) и отправляет синхронный HTTP-запрос к ML-сервису с передачей списка тегов интересов. ML-сервис возвращает упорядоченный список идентификаторов рекомендуемых проектов. View-функция загружает соответствующие проекты из базы данных и передаёт их в шаблон в указанном порядке.

В случае недоступности ML-сервиса активируется резервный алгоритм: поиск проектов по совпадению тегов технологий с тегами интересов пользователя через Q-запросы Django ORM. Резервный режим обеспечивает работоспособность вкладки «Рекомендации» при любом состоянии ML-сервиса без уведомления пользователя о переключении.

3.4.4. Управление заявками

Подача заявки реализована в двух вариантах: быстрое действие с карточки каталога (HTMX POST без перехода на отдельную страницу) и развёрнутая форма на странице проекта с полем мотивационного письма. Для предотвращения дублирования применяется метод get_or_create Django ORM: если заявка уже существует, возвращается существующая запись без создания дубликата.

Жизненный цикл заявки включает три состояния: SUBMITTED (подана), ACCEPTED (принята), REJECTED (отклонена). Переход между состояниями управляется через модуль transitions, проверяющий допустимость перехода перед выполнением. При принятии заявки автоматически пересчитывается число принятых участников проекта. Студент может отозвать заявку в статусе SUBMITTED или обновить мотивационное письмо через страницу редактирования с серверной валидацией длины.

3.5. Особенности функционала

3.5.1. Каталог проектов (/projects/)

Основной интерфейс студента предоставляет каталог с четырьмя вкладками. Вкладка «Все проекты» отображает полный список с фильтрацией по технологиям (tech_tags), размеру команды (team_size) и текстовым поиском (q). Вкладка «Рекомендации» отображает проекты, подобранные ML-сервисом на основе интересов из профиля. Вкладка «Закладки» отображает сохранённые проекты с оптимистичным обновлением UI. Вкладка «Мои заявки» показывает поданные заявки со статусами SUBMITTED / ACCEPTED / REJECTED.

Фильтрация реализована через HTMX-обновление сетки карточек с debounce 350 мс. Атрибут hx-push-url обновляет URL без перезагрузки страницы, что обеспечивает корректную работу кнопки «Назад» и возможность поделиться ссылкой с выбранными фильтрами. При наличии заголовка HX-Request в запросе view возвращает только HTML-фрагмент сетки, иначе — полную страницу с базовым шаблоном.

3.5.2. Кабинет заказчика (/projects/)

Кабинет заказчика организован в пяти вкладках. Вкладка «Сводка» содержит агрегированные метрики: число опубликованных проектов, количество заявок по статусам, свободные места в командах. Метрики вычисляются через аннотации Django ORM (Count, F-выражения) без избыточных запросов к базе данных. Вкладка «Мои проекты» отображает список проектов с аннотированными счётчиками ожидающих заявок (pending_count) и статус-индикаторами комплектования команд.

Вкладки «Статьи» и «Сотрудники» интегрированы с eLIBRARY/Scopus API и отображают публикации и профили сотрудников кафедры. Вкладка «Граф соавторства» реализует интерактивную визуализацию на Vis.js 9.1.9 с ленивой инициализацией: Canvas создаётся только при первом открытии вкладки, что исключает неиспользуемые вычисления. Все маршруты кабинета защищены декораторами login_required и проверкой роли заказчика.

3.5.3. Подача заявки (/projects/
/)

Для студентов реализованы два варианта подачи заявки. Быстрое действие с карточки каталога выполняется через HTMX POST без перехода на отдельную страницу — возвращается партиал с обновлённым состоянием кнопки. Развёрнутая форма на странице проекта содержит поле мотивационного письма с минимальной длиной 30 символов, счётчик символов с визуальной индикацией (смена цвета при достижении порога) и серверной валидацией. HTMX submit возвращает партиал apply_action_detail.html, обновляя только блок действий без перезагрузки страницы.

Для заказчиков страница проекта отображает список поступивших заявок с фильтрацией по статусу и пагинацией (PAGE_SIZE = 20). Действия принятия и отклонения реализованы через модуль transitions, обеспечивающий валидность переходов между состояниями. При принятии заявки автоматически пересчитывается число принятых участников проекта. Студент может отозвать заявку в статусе SUBMITTED или обновить мотивационное письмо через страницу редактирования.

3.5.4. Форма проекта (/projects/create/)

CRUD-интерфейс для заказчиков поддерживает создание и редактирование карточки проекта с полями: название, описание, теги технологий, размер команды, научный руководитель, тип источника (REGULAR или INITIATIVE). Серверная валидация проверяет теги по регулярному выражению: тег должен начинаться с буквы или цифры; допустимые символы — буквы, цифры, дефис, точка, плюс и знак решётки; длина от 2 до 50 символов.

При изменении поля «Размер команды» выполняется пересчёт числа оставшихся свободных мест с учётом уже принятых участников. Новые теги технологий, введённые заказчиком впервые, попадают в очередь модерации ЦППРП и отображаются как «ожидающие» до одобрения модератором. После одобрения тег становится частью общего справочника и доступен другим заказчикам.

3.5.5. Кабинет ЦППРП (/cpprp/)

Административный интерфейс организован в пяти вкладках. Вкладка «Сводка» отображает агрегированную статистику платформы: число проектов по статусам, количество заявок, число активных студентов. Вкладка «Заявки» предоставляет фильтрацию по статусу с пагинацией и поддерживает массовые операции через модуль transitions. Вкладки «Дедлайны» и «Шаблоны документов» реализуют полный CRUD для соответствующих сущностей.

Вкладка «Экспорт» генерирует CSV-файл с кодировкой UTF-8 BOM для корректного открытия в Microsoft Excel. Кабинет включает управление очередью новых технологических тегов — сотрудник ЦППРП одобряет или отклоняет теги, предложенные заказчиками при создании проектов. Все маршруты кабинета защищены декоратором moderator_required, исключающим несанкционированный доступ.

3.5.6. Профиль (/profile/)

Страница профиля обеспечивает управление персональными данными: отображаемое имя, биография (поле bio, минимальная длина 10 символов) и список интересов (до 20 тегов, каждый

от 2 до 50 символов). Клиентская валидация bio реализована через JavaScript-счётчик с визуальной индикацией при превышении 450 символов. Теги интересов проверяются по тому же регулярному выражению, что и теги технологий в форме проекта.

Для пользователей с ролью «студент» страница профиля дополнена статистическим блоком: число поданных заявок, число проектов в закладках, число предложенных инициативных проектов. Интересы из профиля используются ML-сервисом в качестве входных данных для формирования персонализированных рекомендаций. Изменение набора интересов немедленно влияет на результаты вкладки «Рекомендации» при следующем её открытии.

3.6. Обоснование выбора метода организации входных и выходных данных

Входные данные поступают от пользователей через HTML-формы (метод POST) и URL-параметры (метод GET). HTMX-запросы передают данные в виде form-encoded содержимого. Все входные данные валидируются на стороне сервера: теги интересов и технологий — по регулярному выражению (корректность символов, длина от 2 до 50 символов); мотивационное письмо — на минимальную длину не менее 30 символов; поля профиля — на минимальный и максимальный размер. Клиентская предварительная валидация реализована через атрибут minlength и JavaScript-счётчики с визуальной обратной связью.

Внутренние структуры данных основаны на Django ORM-моделях: Project, Application, TechnologyTag, UserProfile, Bookmark, Deadline, DocumentTemplate. Для агрегации статистики в дашбордах используются аннотации Count() и F()-выражения, что позволяет формировать сводные показатели без избыточных запросов к базе данных. Хранение осуществляется в PostgreSQL 16 с индексами на часто фильтруемые поля (status, tech_tags).

Выходные данные представлены в нескольких форматах в зависимости от типа запроса. Обычные запросы возвращают полные HTML-страницы. HTMX-запросы возвращают HTML-фрагменты (partials), подставляемые в указанный DOM-элемент без перезагрузки. Операции с закладками возвращают JSON-ответ, обрабатываемый JavaScript для оптимистичного обновления UI. Административный экспорт генерирует CSV-файл с заголовком UTF-8 BOM. Обмен данными с ML-сервисом рекомендаций выполняется через REST API в формате JSON.

4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

4.1. Ориентировочная экономическая эффективность

Внедрение цифрового ассистента студента позволит сократить временные затраты на обработку заявок и модерацию проектов в среднем на 30-40% за счет автоматизации рутинных процессов. Ожидается снижение количества ошибок при обработке данных на 25% благодаря встроенным механизмам валидации и стандартизированным формам ввода. Система рекомендаций на основе ML-алгоритмов повысит эффективность подбора проектов студентами, что может увеличить процент успешных заявок на 15-20%.

4.2. Предполагаемая потребность

Целевая аудитория системы включает:

1. Студентов НИУ ВШЭ (2-4 курсы ФКН) — около 1700 пользователей в пиковые периоды набора проектов
2. Преподавателей и внешних заказчиков — до 200 активных пользователей
3. Модераторов ЦППРП — 5-10 административных сотрудников

Основные сценарии использования:

- Поиск и подбор проектов с персонализированными рекомендациями
- Управление жизненным циклом проектов (создание, модерация, публикация)
- Администрирование и аналитика проектной деятельности
- Автоматизированная обработка заявок и формирование отчетов

Потребность обусловлена необходимостью цифровизации процессов проектной деятельности и устранением узких мест в существующих решениях (ручная обработка данных, отсутствие интеграции между системами).

4.3. Экономические преимущества разработки по сравнению с отечественными и зарубежными аналогами

Ключевые преимущества системы:

1. Полная интеграция с образовательным процессом НИУ ВШЭ (SSO, ролевая модель, специфичные workflow)
2. Автоматизированная система рекомендаций проектов с ML-компонентом
3. Комплексный инструментарий для модерации и администрирования проектной деятельности
4. Оптимизированные интерфейсы для всех категорий пользователей
5. Поддержка полного жизненного цикла проектов — от инициации до завершения

Сравнение с аналогами:

Функции	«ИПС ФЭН»	«Ярмарка ЭПП»	«Google Sheets + Forms»	**`project-name`**
«ML-рекомендации проектов»	-	-	-	+
«Граф соавторства»	-	-	-	+
«Инициативные проекты»	-	+	-	+
«Кабинет модератора»	+	+	-	+
«Экспорт в CSV с BOM»	-	-	-	+
«Интеграция с SSO НИУ ВШЭ»	+	+	-	+
Итого	**2**	**3**	**0**	**6**

Таблица 1. Сравнение функциональных характеристик аналогов

ПРИЛОЖЕНИЕ 1. СПИСОК ИСПОЛЬЗУЕМОЙ ЛИТЕРАТУРЫ

1. ГОСТ 19.101-77: Виды программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
2. ГОСТ 19.102-77: Стадии разработки. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
3. ГОСТ 19.103-77: Обозначения программ и программных документов. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
4. ГОСТ 19.104-78: Основные надписи. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
5. ГОСТ 19.105-78: Общие требования к программным документам. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
6. ГОСТ 19.106-78: Требования к программным документам, выполненным печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
7. ГОСТ 19.201-78: Техническое задание. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
8. ГОСТ 19.301-79: Программа и методика испытаний. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
9. ГОСТ 19.401-78: Текст программы. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
10. ГОСТ 19.404-79: Пояснительная записка. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
11. ГОСТ 19.505-79: Руководство оператора. Требования к содержанию и оформлению. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
12. ГОСТ 19.603-78: Общие правила внесения изменений. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.
13. ГОСТ 19.604-78: Правила внесения изменений в программные документы, выполненные печатным способом. // Единая система программной документации. – М.: ИПК Издательство стандартов, 2001.

ПРИЛОЖЕНИЕ 2. ССЫЛКИ НА АНАЛОГИ

Приложение	Ссылка
ИПС ФЭН НИУ ВШЭ	https://fen.hse.ru/
Ярмарка ЭПП НИУ ВШЭ	https://epp.hse.ru/
Django (фреймворк)	https://www.djangoproject.com/
HTMX (библиотека)	https://htmx.org/
Tailwind CSS	https://tailwindcss.com/v2/
Vis.js (визуализация графов)	https://visjs.github.io/vis-network/
FastAPI (ML-сервис)	https://fastapi.tiangolo.com/

Дата обращения: 29.04.2026.

ПРИЛОЖЕНИЕ 3. ТЕРМИНОЛОГИЯ

Термин	**Определение**
SSR (Server-Side Rendering)	Подход к рендерингу веб-страниц, при котором HTML формируется на сервере и отправляется клиенту в готовом виде, в отличие от CSR, где страница собирается в браузере.
HTMX	JavaScript-библиотека, позволяющая выполнять AJAX-запросы и обновлять фрагменты страницы (HTML-партиалы) без полной перезагрузки, декларативно через HTML-атрибуты.
ORM (Object-Relational Mapping)	Технология, обеспечивающая автоматическое преобразование между объектами в коде и записями в реляционной базе данных.
ML (Machine Learning)	Раздел искусственного интеллекта, в котором алгоритмы обучаются на данных для построения предсказательных моделей без явного программирования правил.
SSO (Single Sign-On)	Механизм единой аутентификации, позволяющий пользователю войти один раз и получить доступ к нескольким сервисам без повторного ввода учётных данных.
ЦППРП	Центр проектной и профессиональной работы с партнёрами — структурное подразделение ФКН НИУ ВШЭ, осуществляющее модерацию и администрирование проектной деятельности.
BOM (Byte Order Mark)	Специальная последовательность байт в начале файла, указывающая кодировку и порядок байт. BOM в CSV необходим для корректного открытия файла в Microsoft Excel.
Fallback	Резервный механизм или сценарий поведения системы, активируемый при недоступности основного компонента.
Дедлайн	Установленный срок выполнения задачи или этапа проекта, по истечении которого результат считается просроченным.

ЛИСТ РЕГИСТРАЦИИ ИЗМЕНЕНИЙ

Лист регистрации изменений									
Номера листов (страниц)					Всего листов (страниц в докум.)	№ документа	Входящий № сопроводительного докум. и дата	Подп.	Дата
Изм.	Измененных	Замененных	Новых	Аннулированных					